AI 协作日志 W8

文件命名：[姓名][学号]AI 协作日志 W8.docx

姓名：王梓健

学号：2024150200

使用的 AI 工具及方式：Codex（AI 编程助手）。先用自然语言描述“把 W7 render() 里的循环绑定改成父容器 ul 上的事件委托”，让它给出重构骨架；再由我逐行审查 closest('li') 的空值防御与 [data-action="delete"] 的分流判断，并自己在浏览器里用 console.log 与逐项点击验证。

Git 仓库链接：https://github.com/kartwatt-cpu/web-week8（公开仓库，含 feature/v2-event-delegation 分支与合并记录）

任务：完成 V2.0 的事件委托重构，并完成一次 AI-2 调试挑战。包括：

1. 用事件委托重构列表交互（createElement + dataset.id，删除/切换统一交给父容器，处理边界防御）
2. 诊断自己生成的代码或提供代码的缺陷，按五步法留下协作记录（假设、修复、两种测试结果）
3. 禁用 AI，独立完成一次“编辑”分支的迁移（代码或流程说明）

一、任务 1：事件委托重构（两句话记录）

任务要求：将 W7 的循环绑定改为父容器 ul 上的事件委托。使用 e.target.closest('li') 找到任务，用 [data-action="delete"] 判断删除操作，并处理 null 边界。

完整代码在 Git 仓库中验收。

我让 AI 帮我做了：把 W7 render() 里“每条 li 和每个按钮各自 addEventListener”的循环绑定，改成挂在 ul 上的一个事件委托骨架——li 用 createElement + dataset.id 记住自己对应哪条 todo，任务文字放进 span，删除按钮加 data-action="delete"，点击统一交给 ul 的监听器，用 e.target.closest('li') 定位任务、用 e.target.closest('[data-action="delete"]') 判断删除，并补了 closest 返回 null 时的空值防御（if (!item) return）与“删除 / 切换”的 if / else 互斥分流。

我做了：逐条审查并动手改了两处关键边界——①确认 closest('li') 的 null 防御必须写在读 item.dataset.id 之前，否则点列表空白处会直接崩；②把 toggleTodo(id) 收进 else 分支（或删除后 return），让“删除”和“切换”真正互斥，不会删完又接着走 toggleTodo；③在浏览器里逐项点“任务文字 / 删除按钮 / 列表空白 / 动态新增”验证，确认删除和切换各自只触发一次、空白处不报错。

二、任务 2：诊断缺陷代码（完整的五步法）

任务要求：可以使用自己遇到过的缺陷代码，也可以使用下面的缺陷代码。按五步法留下缺陷诊断修复的调试证据。

```
list.addEventListener('click', (e) => {
  if (!(e.target instanceof Element)) return;
  const item = e.target.closest('li');
  if (!item || !list.contains(item)) return;
  const id = item.dataset.id;
  if (!id) return;
  const delBtn = e.target.closest('[data-action="delete"]');
  if (delBtn && item.contains(delBtn)) {
    deleteTodo(id);
  }
  toggleTodo(id);
});
```

五步法核心记录（必须用自己的话）

步骤	我的记录
① 观察现象（提取证据）	我在 `deleteTodo` 和 `toggleTodo` 里加了 `console.log`。 点击「删除」按钮后，控制台打印了：[delete] 1739827200000，紧接着又打了 [toggle] 1739827200000。 **页面实际表现：这条任务被删掉了、计数更新，页面没报错，看起来"一切正常"。**
② 描述上下文（明确问题）	出问题的位置在：if (delBtn && item.contains(delBtn)) { deleteTodo(id); } 的右花括号后面没有 else 兜住，代码继续往下走到了 toggleTodo(id)。 **预期行为是：点「删除」只调用 deleteTodo，toggleTodo 不该被调用，控制台只打一条。** 实际行为是：deleteTodo 和 toggleTodo 都被调用，控制台连打两条。
③ 先猜原因（严禁抄 AI）	我的假设是：删除分支没有 else 兜住（也没有 return），执行完继续往下走，toggleTodo(id) 被"顺带"执行，把本应二选一的操作变成了"先删再切"。 **我的依据是：控制台里 [delete] 和 [toggle] 两条都打了，而代码里 deleteTodo(id) 之后紧跟的就是 toggleTodo(id)，两者之间没有任何 else 或 return 把它们隔开。**
④ 实施修复	我改动了：把 toggleTodo(id) 移进 else 分支——命中删除就只删除，其余情况才切换（等价地，也可以删除后直接 return）。 **修改理由是：删除与切换本来就是互斥的，不是"先删再切"。原来的 if 后面没有 else，就等于删除之后又把切换走了一遍，这是"操作分流"不完整；加上 else 后两条分支才真正互斥。**

⑤ 运行验证	见下方"统一测试表"（5 个场景全部跑通才算验证完成）。
追问（如果页面没崩、看起来还能用，我凭什么说它有问题？）： 答：页面没崩、看起来还能用，不代表代码是对的。它的问题是"操作分流不完整"：删除分支后面没有 else，toggleTodo(id) 被多余执行了一次。虽然这一次因为任务已被删掉、toggleTodo 里的 map 找不到对应 id 而没有可见后果，但它多做了一次 save() + render()，一旦删除与切换的边界重叠就可能悄悄改错状态。这种不报错的静默逻辑失败，只能靠控制台打印记录这种运行证据暴露出来，光看页面表现是看不出来的。	

统一测试表

测试场景	实际结果
点击任务文字，只切换	通过。控制台只打一条 [toggle]，该条的 completed 翻转，其余项不变。
点击删除按钮，只删除	通过。控制台只打一条 [delete]，该条被移除，没有多打 [toggle]。
点击删除按钮里的文字/图标，正确删除	通过。closest('[data-action="delete"]') 从按钮内的文字/图标向上找到按钮，正确删除对应那条。
点击列表空白处，不报错	通过。closest('li') 返回 null，被 if (!item) return 挡住，控制台无报错。
动态新增任务后，能正常交互	通过。新增一条后无需重新绑定监听器，直接点击即可切换/删除，委托生效。

控制台截图（1-2 张）：

下面两张是控制台的真实打印记录：图 1 是原错误场景（修复前点「删除」），图 2 是边界场景（点列表空白处）。修复后点「删除」只打印一行 [delete]，见统一测试表。



图 1　修复前：点「删除」按钮，[delete] 与 [toggle] 连打两条——第二条就是被“顺带”执行的 toggle



图 2　边界场景：点列表空白处，closest('li') 返回 null 被 if (!item) return 挡住，控制台不报错

AI 在调试中帮了什么？

我问了 AI：我怀疑删除分支少了 else，导致 toggleTodo 被多执行，请你只针对“操作分流”这个点确认一下，别直接给整段重写。

它帮到了，因为：它确认了“缺 else / 分流不完整”这个判断，并指出在委托架构里应该用条件分流而不是 stopPropagation；但“没报错为什么也算错”这句结论，是我自己从控制台的双打印记录里推出来的，不是它给的。

三、任务 3：独立迁移（本任务严禁使用 AI）

任务要求：不依赖 AI，独立为列表增加一个“编辑”分支。重点是自己手写。

核心点和流程说明（简写）

核心流程（全程手写，未用 AI）：

① render() 里在每条 li 再加一个「编辑」按钮：type="button"、data-action="edit"、文字写「编辑」，并带 aria-label；

② 父容器 ul 的事件委托改成 if / else if / else 三段：删除 → 编辑 → 切换，三个分支互斥；

③ 新增 editTodo(id)：先用 todos.filter 找到当前文本，prompt('修改任务', current.text) 读新文本；点「取消」或 trim() 后为空就 return 不动数据；否则 todos = todos.map(function (t) { return t.id === id ? Object.assign({}, t, { text: text }) : t; })，结尾固定 save(); render();；

④ 关键点：编辑也走「改数据 → save() → render()」三步、不改原数组；三个分支互斥，编辑完不会再落到 toggleTodo 上，避免重演任务 2 的静默失败。

我是否使用了 AI？

☑ 没有，完全独立完成

☐ 用了（如果用了，用在哪里，能不能脱离 AI 完成？）

独立验证结果（一句话）

我测试了【点击「编辑」按钮 → 输入新文本 → 确认】场景，结果是该条文字就地更新（前后空格被 trim）、其余项与完成状态不变，刷新后新文本仍在（save 生效），且编辑分支没有顺带触发切换。

四、一句话总结

本周的调试训练中，你最大的收获是什么？

最大的收获是明白“没报错不等于没 bug”：事件委托里一个漏掉的 else 就能让删除和切换被同时触发，页面照样能用、控制台也不报错，只有靠“在 deleteTodo 和 toggleTodo 里双打 console.log”这种运行证据才能把它揪出来。所以现在调试的顺序变了——先自己复现、留下打印记录、形成假设，再让 AI 帮忙确认，最后用两种测试结果验收。